

## Lab 1 — Setting up Google Colab

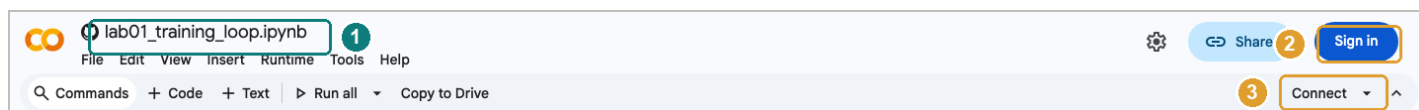
Everything runs in your browser. Nothing to install.

**What you need:** a web browser and a **free Google account** (a personal @gmail.com is fine — some hospital and university accounts block Colab, so if yours does, use a personal one). **You do not need:** Python, PyTorch, an install, or a GPU. *Colab* is a free Google service that runs Python notebooks on Google's computers; your laptop only shows the page.

### Step 1 — Open the Lab 1 notebook

Click the Lab 1 link (on the course page, or type it in). It opens Colab with the notebook already loaded — there is nothing to download first.

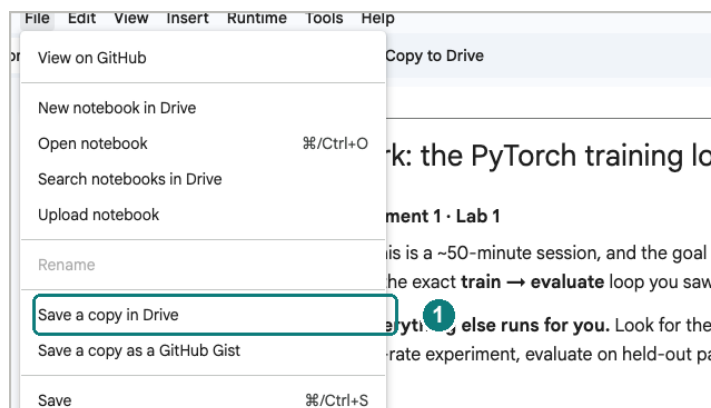
[https://colab.research.google.com/github/indigobio/msacl\\_dl\\_course/blob/main/student\\_pack/labs/lab01\\_training\\_loop.ipynb](https://colab.research.google.com/github/indigobio/msacl_dl_course/blob/main/student_pack/labs/lab01_training_loop.ipynb)



- (1) Check the file name reads `lab01_training_loop.ipynb` — that is the right notebook.
- (2) **Sign in** (top right) with your Google account. Until you do, you can read the notebook but not run it.
- (3) **Connect** (top right) starts your free machine. It usually connects on its own the first time you run a cell; if it says *Connect*, click it and wait for a green tick.

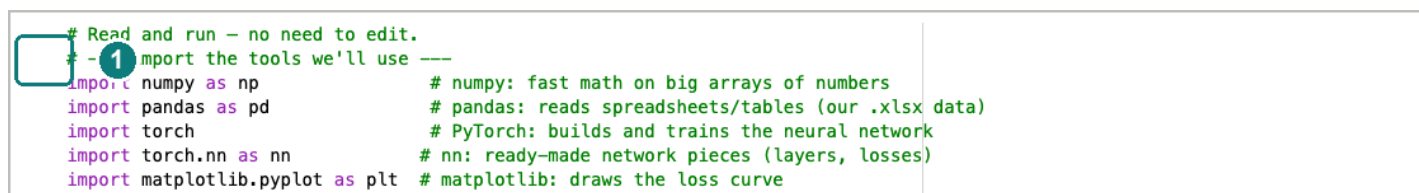
### Step 2 — Save your own copy (do this before you type anything)

The link opens a *read-only* copy. Use **File** → **Save a copy in Drive** and work in that copy, or your edits will be lost when you close the tab. Colab opens the copy in a new tab, named *Copy of lab01\_training\_loop.ipynb* — **that is the tab you want to work in**.



### Step 3 — Run a cell

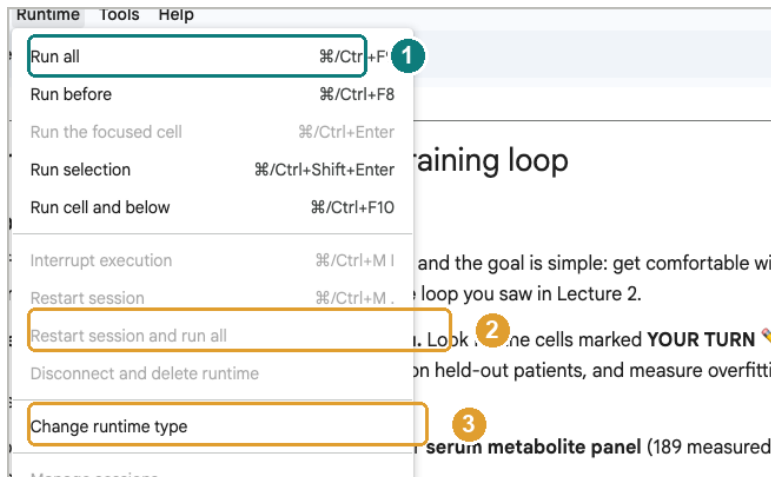
A notebook is a list of *cells*. A cell holds either text or code. Click a code cell, then press **Shift + Enter** — or hover over the (1) bracket at its left, which turns into a ► button, and click it.



- Run the cells **in order, top to bottom**. Later cells depend on earlier ones.
- While a cell runs the bracket shows a spinner; when it finishes you get a number, e.g. [1].
- **Run the first code cell, “Setup”, before anything else**. It installs the course packages and downloads the lab's data for you — about a minute, ending with `lab01 is ready`. You never download or copy data files yourself.
- Only the cells marked **YOUR TURN** (the pencil icon) need editing. Run all the others exactly as they are.

## Step 4 — If something gets tangled

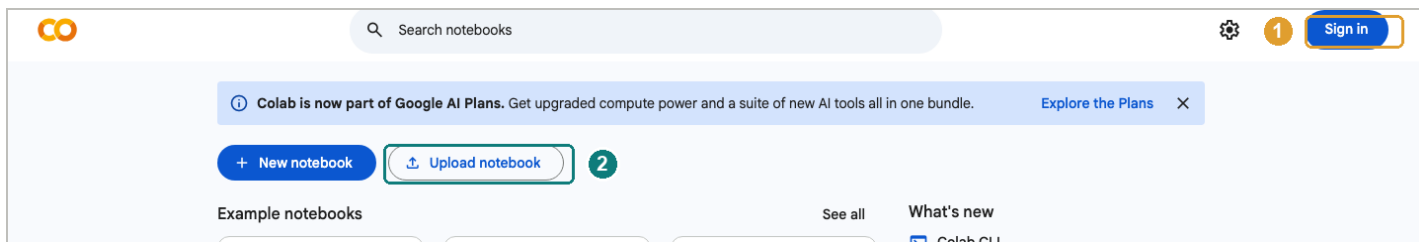
The **Runtime** menu is the fix for almost everything.



- (1) **Run all** — runs the whole notebook from the top. Good after you fix a cell.
- (2) **Restart session and run all** — throws away every variable and starts clean. This is the answer to “it worked a minute ago”. You lose nothing but the running state; your typed code stays.
- (3) **Change runtime type** — where you would pick a **T4 GPU**. **Lab 1 does not need one**; the default CPU is plenty and connects faster. You will use this in Lab 2.

## Step 5 — If the link does not open

Go to [colab.research.google.com](https://colab.research.google.com) directly, sign in (1), then use **File** → **Open notebook** → **GitHub** and paste [indigobio/msacl\\_dl\\_course](https://github.com/indigobio/msacl_dl_course). Or use **Upload notebook** (2) if an instructor hands you the .ipynb file on a USB stick.



## When things go wrong

### What you see

### What to do

“Sign in” keeps reappearing, or Colab will not load

Your institutional Google account may block Colab. Sign in with a personal [@gmail.com](mailto:@gmail.com) instead.

The setup cell says “Restarting now”, then “session restarted” / “crashed”

Expected, once per session: the course packages replaced ones Colab had already loaded. Wait for it to reconnect, then run the **setup cell** again.

`AttributeError` mentioning `numpy` or `_multiarray_umath`  
`NameError / ... is not defined`

The session was not restarted after setup. **Runtime** → **Restart session**, then run the setup cell again.

You skipped a cell, or ran them out of order. **Runtime** → **Restart session and run all**.

A cell has been running for minutes

Normal for training cells. If it is stuck, **Runtime** → **Interrupt execution**, then restart and run all.

“Cannot connect to runtime” / disconnected

Free Colab disconnects after a while idle. Click **Connect** and **Runtime** → **Run all**; you lose only the running state.

An **assert** cell failed

That is the notebook checking your answer — it is meant to catch you. Read the message, fix the marked line, re-run that cell. The **Code Hint Sheet** lists the options for every blank.

You edited the wrong tab and lost your work

You were in the read-only original. Redo **File** → **Save a copy in Drive** and check the tab title starts with Copy of.

*Screenshots are of the Google Colab interface, captured 22 September 2026, with callout numbers added for this sheet. Colab is a Google product and its appearance changes from time to time — if a button has moved, the menu path in the text is still the one to follow. A clickable version of this sheet is [lab01\\_colab\\_setup.html](#).*



## Lab 1 — Code Hint Sheet

Your First Network: the PyTorch training loop

Stuck on a **YOUR TURN** cell? Each line you need is below with a few options. Pick one, type it in, and **run the cell** — the built-in `assert` checks will tell you whether it is right. Only the two marked cells need editing; run everything else as-is.

---

**Step 4 — the training loop.** Write these five lines *in order* inside the `for epoch` loop (Lecture 2: forward → loss → zero → backward → step).

**1. Forward — make a guess.**

- (a) `pred = model(X_train)`
- (b) `pred = loss_fn(X_train)`
- (c) `pred = model.guess(y_train)`

**2. Loss — how wrong?**

- (a) `loss = loss_fn(pred, y_train)`
- (b) `loss = model(pred, y_train)`
- (c) `loss = loss_fn(X_train, y_train)`

**3. Zero — clear last step's gradients.**

- (a) `optimizer.zero_grad()`
- (b) `optimizer.reset()`
- (c) `model.zero_grad(loss)`

**4. Backward — backprop the blame.**

- (a) `loss.backward()`
- (b) `optimizer.backward()`
- (c) `model.backward(loss)`

**5. Step — nudge every weight.**

- (a) `optimizer.step()`
- (b) `optimizer.update()`
- (c) `loss.step()`

---

**Step 6 — the learning rate: find all three regimes.** You are given `lr_ok = 0.01`. Pick one rate that is *too small* (the loss barely moves in 150 epochs) and one that is *too big* (the loss climbs). The plot shows all three curves.

**1. A learning rate that is TOO SMALL.**

- (a) `lr_too_small = 0.0001` (100× smaller: loss 0.277 → 0.213, still crawling)
- (b) `lr_too_small = 0.003` (smaller, but still trains well: ends at 0.017 — not a crawl)
- (c) `lr_too_small = 0.1` (that is *bigger* than `lr_ok`)

**2. A learning rate that is TOO BIG.**

- (a) `lr_too_big = 5.0` (huge steps overshoot: loss 0.277 → 0.487, it rose)
- (b) `lr_too_big = 0.1` (bigger, but it wobbles and still ends lower, at 0.073)
- (c) `lr_too_big = 0.001` (that is *smaller* than `lr_ok`)

**Then explore:** the next cell has no blank. Change `lr_try`, re-run it, and each run is added to the plot. Try 0.001, 0.003, 0.03, 0.1, 0.3, 1.0 and compare the final losses.

---

**Step 7 — evaluate on the test set.** Predict without learning, threshold at 0.5, then measure the fraction correct.

**1. Predict without gradients.**

- (a) with `torch.no_grad():` (then, indented) `probs = model(X_test)`
- (b) with `torch.grad():` (then, indented) `probs = model(X_test)`
- (c) `probs = model.no_grad(X_test)`

**2. Turn probabilities into 0/1 predictions.**

- (a) `predicted = (probs > 0.5).float()`
- (b) `predicted = probs.round(threshold=0.5)`
- (c) `predicted = probs.argmax()`

**3. Accuracy — fraction that match the label.**

- (a) `test_acc = (predicted == y_test).float().mean().item()`
  - (b) `test_acc = (predicted - y_test).mean()`
  - (c) `test_acc = accuracy(predicted, y_test)`
-

**Step 8 — did it overfit? compute the training accuracy the same way.** Mirror the Step 7 evaluation, but on `X_train/y_train`, then compare to `test_acc`.

**1. Training accuracy (same recipe as Step 7, on the training data).**

- (a) with `torch.no_grad()`: `probs_train = model(X_train)` then  
    `predicted_train = (probs_train > 0.5).float()`  
    `train_acc = (predicted_train == y_train).float().mean().item()`
- (b) `train_acc = (model(X_test) == y_test).float().mean()` (that's the *test* set, not train)
- (c) `train_acc = loss_history[-1]` (that's a loss, not an accuracy)

**2. Overfitting means the model scores HIGHER on data it has seen. Fill the comparison.**

- (a) `overfitting = train_acc > test_acc` (train beats test  $\Rightarrow$  overfit)
- (b) `overfitting = train_acc < test_acc` (backwards — and the assert will fail)
- (c) `overfitting = train_acc == test_acc` (they're not equal here)

---

*Answers self-check in the notebook: if the **assert** cells print their success message, you chose right. The loss should fall steeply and then flatten; the too-big learning rate makes the loss rise instead; test accuracy should land around **0.72–0.77** (coin-flip baseline is 0.50); and training accuracy should be clearly higher than test accuracy (a train/test gap of roughly **0.20–0.25**) — that gap is overfitting.*

## Lab 2 — Code Hint Sheet

### A 1D CNN on Real Clinical Spectra

Stuck on a **YOUR TURN** cell? Each blank you need is below with a few options. Pick one, type it in, and **run the cell** — the built-in `assert` checks will tell you whether it is right. Only the five marked cells need editing; run everything else as-is.

**Step 3 — the training loop.** Write these five lines *in order* inside the batch loop (Lecture 2: forward → loss → zero → backward → step) — same recipe as Lab 1, reused here for spectra.

**Blank 1 — the five-line loop body.**

- (a) `pred = model(X_batch)` then `loss = loss_fn(pred, y_batch)` then `optimizer.zero_grad()` then `loss.backward()` then `optimizer.step()`
  - (b) `pred = loss_fn(X_batch)` then `loss = model(pred, y_batch)` ...
  - (c) `loss.backward()` then `pred = model(X_batch)` then `optimizer.step()` ...
- (a) is the Lecture 2 order; (b) swaps what `model` and `loss_fn` do; (c) backs up before making a guess — there is nothing to backprop yet.

**Step 4 — the first conv block.** One input channel (a raw spectrum), 8 filters, a 15-bin-wide filter, and padding so the length is unchanged (Lecture 5's code slide).

**Blank 2 — `self.conv1`.**

- (a) `nn.Conv1d(in_channels=1, out_channels=8, kernel_size=15, padding=7)`
- (b) `nn.Conv1d(in_channels=8, out_channels=1, kernel_size=15, padding=7)`
- (c) `nn.Linear(in_features=1, out_features=8)`

**Step 4 — the second conv block.** Mirror block 1's pattern, but now reading the 8 channels block 1 produced and turning them into 16.

**Blank 3 — `self.conv2`, `self.relu2`, `self.pool2`.**

- (a) `nn.Conv1d(in_channels=8, out_channels=16, kernel_size=15, padding=7)`  
then, same as block 1: `nn.ReLU()` and `nn.MaxPool1d(4)`
- (b) `nn.Conv1d(in_channels=16, out_channels=8, kernel_size=15, padding=7)`
- (c) `nn.Conv1d(in_channels=8, out_channels=16, kernel_size=1, padding=0)`

**Step 4 — the input size of the first Linear layer.** The `Conv1d` layers keep the length unchanged (that's the padding); each `MaxPool1d(4)` divides the length by 4, and there are **two** of them; `Flatten` then stacks all 16 channels into one vector. So:  $16 \times (6000 \div 4 \div 4)$ .

**Blank 4 — `self.fc1`.**

- (a) `self.fc1 = nn.Linear(16 * (6000 // 4 // 4), 32)`
  - (b) `self.fc1 = nn.Linear(16 * 6000, 32)`
  - (c) `self.fc1 = nn.Linear(8 * 375, 32)`
- (a) is right: two `MaxPool1d(4)`s take  $6000 \rightarrow 1500 \rightarrow 375$ , and 16 channels give  $16 \times 375 = 6000$  features. (b) forgets the pooling shrank the length; (c) uses the wrong channel count (8, not 16) — both make `fc1` the wrong size and the forward pass will error.

**Step 5 — pick a batch size.** A middle-ground value for 590 training spectra (Lecture 4's Goldilocks range).

**Blank 5 — `CNN_BATCH_SIZE`.**

- (a) `CNN_BATCH_SIZE = 32`
  - (b) `CNN_BATCH_SIZE = 1`
  - (c) `CNN_BATCH_SIZE = 590`
- (a) and other values in roughly 8–64 all pass the check and train fine — (b) updates extremely often but painfully slowly, (c) is the whole training set at once, no mini-batching at all.

**Step 8 — commit to a threshold.** From your Step 7 exploration, pick a cut that catches at least *half* the resistant cases (recall  $\geq 0.5$ ). On this trained model recall stays 0 near 0.5 and climbs only for low cuts.

**Blank 6 — `chosen_threshold`.**

- (a) `chosen_threshold = 0.05`
  - (b) `chosen_threshold = 0.5`
  - (c) `chosen_threshold = 0.9`
- (a) reaches recall  $\approx 0.67$  — it clears the `assert recall_at_cut >= 0.5`. (b) is the default where recall  $\approx 0$ ; (c) is even stricter, still 0. Any low cut that hits recall  $\geq 0.5$  passes — there is no single "right" number, it's the clinical trade-off.

---

*Answers self-check in the notebook: the shape cell confirms `fc1` accepts the right number of features and the forward pass returns one logit per spectrum; the batch-size and threshold cells print their own success messages. Watch the **resistant recall** number, not just accuracy — a model that always says "susceptible" can still score  $\sim 94\%$  accuracy while catching zero real cases.*

## Lab 3 — Code Hint Sheet

Transfer Learning by Fine-Tuning — Teach a Pretrained Generator YOUR Symbol

Stuck on a **YOUR TURN** cell? Each of the three blanks is below with a few options. Pick one, type it in, and **run the cell** — the built-in `assert` checks (and the before/after pictures) will tell you whether it is right. Only the three marked cells need editing; run everything else as-is.

---

**Step 2 — invent your symbol.** Edit the ASCII grid so '#' is ink and '.' is blank. Every row must stay the **same length**, and you want a **bold, centered** shape (roughly 10–40% ink) — thin one-pixel lines are too faint to learn.

**Blank 1 — SYMBOL (the drawing).**

- (a) A bold, centered shape — e.g. a heart with two round lobes on top and a point at the bottom, its widest rows a solid run of about a dozen '#'. Any heart / star / letter / initial drawn this boldly passes.
  - (b) A single thin line (one '#' per row) — too little ink (fails the  $0.03 < \text{ink} < 0.6$  check, and too faint to learn).
  - (c) Every cell set to '#' (the whole grid filled) — too much ink (fails the same check; a solid block has no shape to recognize).
- (a) is the goal. (b) and (c) trip the ink-amount `assert`; rows of unequal length also error when the grid is parsed.

---

**Step 3 — the diffusion training objective.** The model was handed a noisy image and guessed the noise (`noise_pred`); the truth is `noise`. Score how far the guess is from the truth with **mean-squared error** and store it in `loss`.

**Blank 2 — the loss line.**

- (a) `loss = F.mse_loss(noise_pred, noise)`
  - (b) `loss = F.mse_loss(noise_pred, noisy)`
  - (c) `loss = F.cross_entropy(noise_pred, noise)`
- (a) is right — diffusion training makes the *predicted noise* match the *real noise* you added. (b) compares the guess to the noisy *image*, not the noise — wrong target. (c) cross-entropy is for class *labels*, not for matching two continuous noise images — the wrong tool. It does not error (same-shape float targets are read as probabilities), so the scalar/finite/ $\geq 0$  self-checks still pass; you only see it fail *visually* at Step 6, where the **AFTER** grid never becomes your symbol.

---

**Step 4 — the fine-tune learning rate.** You are *nudging* a model that already knows how to draw, not training from scratch — so use a rate **much smaller** than a from-scratch rate. The check is  $0 < \text{FINETUNE\_LR} < 1\text{e-}2$ .

**Blank 3 — FINETUNE\_LR.**

- (a) `FINETUNE_LR = 1e-4`
  - (b) `FINETUNE_LR = 1e-1`
  - (c) `FINETUNE_LR = 1e-6`
- (a) is the sweet spot — big enough to pick up your symbol in  $\sim 100$  steps, small enough not to wreck the pretrained model. (b) is far too big: it blows the model apart and the **AFTER** grid is garbage (and it exceeds the  $< 1\text{e-}2$  check). (c) is so tiny the model barely moves — the **AFTER** samples still look like digits instead of your symbol.

---

*Answers self-check in the notebook: the symbol cell prints its ink % and shape; the objective cell prints a probe loss; and the learning-rate cell prints its value. Step 6 is a **visual** before-vs-after: watch the **AFTER** row — it should unmistakably show the symbol you drew, while the **BEFORE** row still shows the pretrained digits.*



## Lab 4 — Code Hint Sheet

### The Fashion VAE — a Latent Playground + Impostor Detector

You fill in **three** short pieces — a one-line idea that matters (the VAE loss), a threshold plus a prediction, and a creative drawing. Everything else — the model, the encoder/decoder, the reparameterization trick, the training loop, and every plot — runs as-is. Type your answer in and **run the cell**; the built-in `assert` checks give instant feedback.

**Step 2 — the VAE loss.** A VAE trains on **two ideas at once**: `recon_loss` (rebuild the picture) and `kl_loss` (keep the latent cloud tidy near the origin). Both are already computed for you; your one line **combines** them, with the KL term weighted by `beta`.

**Blank 1 — loss (combine the two terms).**

- (a) `loss = recon_loss + beta * kl_loss` — **add** the KL penalty (scaled by `beta`) to the reconstruction. This is a VAE.
  - (b) `loss = recon_loss - beta * kl_loss` — *subtracting* the penalty **rewards** a messy latent; the space stops being sampleable. Wrong sign.
  - (c) `loss = recon_loss * beta * kl_loss` — *multiplying* is not the recipe; when `kl_loss` is near 0 the whole loss collapses to  $\approx 0$ . Wrong.
- (a) is the VAE objective (reconstruction +  $\beta \cdot \text{KL}$ ). The self-check asserts `loss == recon_loss + BETA*kl_loss` (scalar, finite,  $\text{KL} \geq 0$ ), so both the wrong sign (b) and the wrong operator (c) **fail** the assert with a clear message.

**Step 7 — the impostor threshold (+ a prediction).** Reconstruction error is the impostor score: clothes rebuild well (low), unseen digits rebuild badly (high). The rule: put the line at the **90th percentile of the *normal* (clothes) errors**, so about **10%** of real clothes are false alarms.

**Blank 2a — THRESHOLD.**

- (a) `THRESHOLD = np.percentile(normal_errors, 90)` — the 90th percentile of the **normal** errors, exactly the rule.
- (b) `THRESHOLD = np.percentile(impostor_errors, 90)` — wrong array: you would calibrate on the *digits* you are trying to catch, not the clothes.
- (c) `THRESHOLD = normal_errors.mean()` — the mean sits far below the 90th percentile, so roughly half of all clothes would false-alarm.

**Blank 2b — expected\_false\_alarms (predict, then check).**

- (a) `expected_false_alarms = round(0.10 * len(normal_errors))` — by construction the 90th percentile leaves **10%** of normals above the line.
  - (b) `expected_false_alarms = round(0.90 * len(normal_errors))` — confuses the percentile (90) with the false-alarm rate (10%).
  - (c) `expected_false_alarms = 0` — a 90th-percentile cut always flags  $\approx 10\%$  of normals, never 0.
- (a) for both. The prediction `assert` confirms the count you predicted is what the threshold actually produces.

**Step 8 — draw your own impostor.** Edit the 16×16 ASCII grid: `'#'` is ink, `'.'` is blank. The rule is only about **format** — keep **every row exactly 16 characters** and use only `'#'` and `'.'`. The code upsamples it to 28×28 and tells you whether the VAE flags it as an impostor.

**Blank 3 — DOODLE (the drawing).**

- (a) Any shape — the shipped **spiral**, your initials, a squiggle, a molecule sketch — as long as it is a list of **16 rows**, each exactly **16 chars** of only `'#'`/`'.'`. This passes and gets scored against the threshold.
- (b) Rows of *unequal* length, a wrong number of rows, or a stray character like `'x'` or a space — trips the grid sanity-check `assert`.
- (c) An empty grid (all `'.'`) — runs, but there is nothing for the detector to judge; draw *something* thin (lines and text flag easily; a big solid blob may slip through).

*Answers self-check in the notebook: the loss is a finite scalar with the KL term non-negative, the threshold flags about 10% of clothes and separates digits well (AUROC > 0.85), and your doodle parses to (1, 784). The payoff is the tidy VAE latent map (vs. the scattered plain-AE one), the sneaker→boot morph, generated clothes, and your own doodle judged by the detector.*



## Lab 5 — Capstone Code Hint Sheet

An End-to-End MS Mini-Project — three tracks, two blanks each

You do **one** track. Set `TRACK` to 'A', 'B', or 'C', then fill that track's **two** blanks below. Pick an option, type it in, and **run the cell** — the built-in `assert` checks tell you if your choice is structurally sound (shapes, split, frozen / trainable parameters, metrics in `[0, 1]`). They never check accuracy *magnitude*, so a small run still passes. Everything outside the marked blanks runs as-is.

**Wrapping layers in a module (blanks A1 and B1).** Those two blanks ask for a whole little `nn.Module` class — not just the layer list. Keep the skeleton exactly as below (same class **name** the blank asks for) and drop the option-(a) layers into the marked slot:

```
class ImprovedCNN(nn.Module):
    def __init__(self):
        class ROIClassifier(nn.Module):
            super().__init__()
            def __init__(self):
                self.features = <A1 conv layers>
                super().__init__()
                self.head = <A1 linear head>
                self.net = <B1 layers>
            def forward(self, x):
                def forward(self, x):
                    return self.head(self.features(x))
                return self.net(x)
```

Track C's blank C1 is different — it is plain assignment statements that reuse the provided `TransferModel` wrapper, so option (a) there is already complete as written.

---

### TRACK A — Beat the baseline

**Blank A1 — ImprovedCNN.** A *different* 1D-CNN than the Lab 2 baseline: deeper/wider conv blocks, an adaptive pool so the head size is fixed, one logit out.

- (a) `nn.Sequential(Conv1d(1,16,15,pad7),ReLU,MaxPool1d(4),Conv1d(16,32,15,pad7),ReLU,MaxPool1d(4),Conv1d(32,32,15,pad7),AdaptiveMaxPool1d(8),Flatten)` feeding `Linear(32*8,64) → Linear(64,1)`
- (b) copy `BaselineCNN` unchanged (two blocks `1→8→16`, `Linear(16*375,32)→Linear(32,1)`)
- (c) `nn.Linear(6000, 1)` only — no conv layers at all

(a) is right: three wider blocks give a *different* trainable-parameter count (the check is `improved_params != baseline_params`) and still output `(N,1)`. (b) is an exact copy — same param count, so it **fails** the “must differ” check (and doesn’t “improve” anything). (c) throws away the peaks-are-local idea and, on a raw 6000-vector, is a weaker model, not a better CNN.

**Blank A2 — augment.** An m/z-jitter augmentation: nudge the spectrum a few bins along the mass axis and add a little noise. Must keep the shape and *change* the values.

- (a) `shift=randint(1,4); x=torch.roll(x,shift,dims=-1); return x+0.01*randn_like(x)`
- (b) `return x` (no change)
- (c) `return torch.zeros_like(x)`

(a) shifts by 1–3 bins (always nonzero) and adds noise, so it passes both checks (`shape unchanged` and `not torch.allclose(aug, x)`). (b) returns the input untouched — fails the “must change the values” check and teaches the model nothing new. (c) destroys the spectrum entirely (all zeros) — technically “changed,” but it erases every peak, so training collapses.

---

### TRACK B — Peak QC (real peak vs. noise)

**Blank B1 — ROIClassifier.** A small 1D-CNN reading one 256-point ROI `((N,1,256))` and returning one logit `(N,1)`.

- (a) `Conv1d(1,8,7,pad3),ReLU,MaxPool1d(4),Conv1d(8,16,7,pad3),ReLU,MaxPool1d(4),Flatten,Linear(16*16,32),ReLU,Linear(32,1)`
- (b) `Conv1d(1,8,7,pad3),ReLU,MaxPool1d(4),Flatten,Linear(16*16,32),ReLU,Linear(32,2)`
- (c) `Flatten,Linear(256,1)` directly (no conv)

(a) is right: two pools take `256→64→16`, so `Linear(16*16,32)` matches and the output is `(N,1)` (the check is `out.shape == (2,1)`). (b) has *one* pool, so the flattened size is `8*64`, not `16*16` — a shape-mismatch crash — and `Linear(...,2)` outputs two columns, failing the `(N,1)` check (we use one logit + `BCEWithLogitsLoss`). (c) runs but is a bare linear model — allowed to try, yet it ignores the local peak shape the segment is about.

**Blank B2 — pos.weight.** Handle the imbalance (noise outnumbers peaks): up-weight the rare positive class by the `neg:pos` ratio, as a one-element tensor.

- (a) `pos_weight = torch.tensor([n_neg_train / n_pos_train])`  
 (b) `pos_weight = torch.tensor([1.0])`  
 (c) `pos_weight = torch.tensor([n_pos_train / n_neg_train])`  
 (a) is right — noise:peak is about 2.3, so the weight is  $>1$  (the check requires `pos_weight.item() > 1`) and `BCEWithLogitsLoss` then pays more attention to the rare peaks. (b) weights the classes equally — no imbalance handling — and fails the  $>1$  check. (c) is the ratio *upside down* ( $<1$ ), which *down*-weights the already-rare peaks — exactly wrong, and it fails the  $>1$  check too.

---

### TRACK C — New question, same spectra (transfer)

---

**Blank C1 — freeze the body, attach a fresh head.** Reuse the trained `saureus_model.features` as a *frozen* body, add a new head whose last layer outputs *one* logit, wrap in `TransferModel`.

- (a) `transfer_body = saureus_model.features; for p in transfer_body.parameters(): p.requires_grad = False; transfer_head = Sequential(Linear(saureus_model.feat_dim,32),ReLU,Linear(32,1)); transfer_model = TransferModel(transfer_body, transfer_head)`  
 (b) same, but **omit** the `requires_grad = False` loop  
 (c) same, but final layer is `Linear(32, 2)`  
 (a) is right: freezing every body param makes `n_frozen == n_body` and leaves only the head trainable (`n_trainable == n_head`), and the `Linear(...,1)` head gives (N,1) logits — all three checks pass. (b) leaves the body trainable, so `n_trainable` includes the body and the `== n_head` check **fails** (and it isn't transfer learning any more — you'd retrain everything). (c) outputs two columns, failing the (N,1) check (this is a single-logit binary task).

**Blank C2 — TRANSFER\_LR.** A *gentle* learning rate to fine-tune the fresh head on the frozen body — positive and  $\leq 1e-2$ .

- (a) `TRANSFER_LR = 5e-4`  
 (b) `TRANSFER_LR = 0.5`  
 (c) `TRANSFER_LR = 0`  
 (a) is a good small step (smaller than the  $1e-3$  the body was trained with); it passes  $0 < \text{TRANSFER\_LR} \leq 1e-2$ . (b) is *huge* — it fails the  $\leq 1e-2$  check and would blow the head up. (c) is zero — it fails the  $> 0$  check and the head never learns anything.

---

*Every track ends the same way (Lecture 10): report **sensitivity, specificity, and AUROC + the confusion matrix** from **honest\_eval** on the held-out split — never bare accuracy — and read the honest-error-analysis cell before you fill the rubric and your project one-pager.*

# My First Deep-Learning Project — One-Pager

Name: \_\_\_\_\_ Assay / instrument: \_\_\_\_\_

Fill one page for a real project in *your* lab. The goal is a plan you could hand a collaborator — and an honest answer to “do I even need deep learning?” Keep it concrete: one example, real shapes and units, a split with no leakage, and clinical-grade metrics (Lecture 10).

## 1 • Problem

*What decision or measurement would the model make, and who uses the output? E.g. “flag likely-resistant isolates from the MALDI spectrum before culture comes back.”*

## 2 • Data & representation

*Describe one example exactly: shape, units, how many samples, how they are labeled, and how imbalanced. E.g. “one 6000-dim TIC-normalized spectrum; ~700 samples; 5–25% positive; labeled by AST.”*

## 3 • Architecture

*Which model, and why does it fit this representation? E.g. “1D-CNN — peaks are local patterns anywhere along m/z, like Lab 2 / Track A”; or MLP for a tabular panel; or transfer/re-head an existing model (Track C).*

## 4 • Validation plan

*What split (no leakage — split by patient/batch, not by row!), which metrics, and what target? E.g. “stratified 80/20 by patient; sensitivity / specificity / AUROC + confusion matrix;  $\geq 0.90$  sensitivity at a fixed specificity, threshold chosen on a validation set — never a single accuracy number.”*

## 5 • Why not a simpler model?

*What baseline must deep learning beat to be worth it — logistic regression, a single-peak threshold, or always-guess-the-majority-class? If a one-line rule already hits your target, deep learning may be the wrong tool — and saying so is a valid, honest conclusion (the Lecture 10 mindset).*

*Discussion D5: share this page with a neighbor. Pressure-test each other on leakage (could val info leak into training?), the honest majority-class baseline, and the clinical cost of a false negative vs. a false alarm — does your threshold reflect it?*

